
Dynamic Arrays & Linked Lists

- ❑ Dynamic Arrays
- ❑ Singly Linked Lists
- ❑ Doubly Linked Lists

Introduction: dynamic array

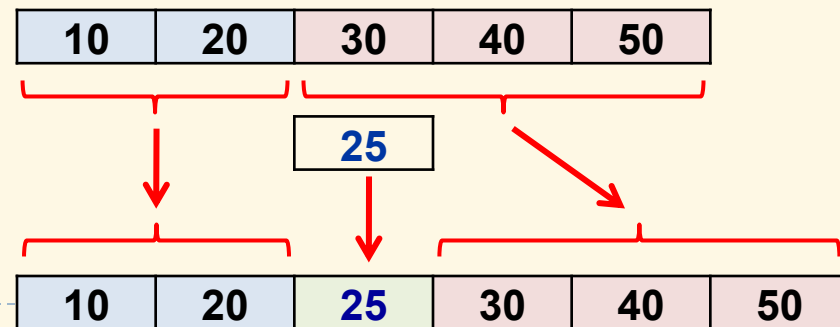
- ▶ Conventional array in C has fix number of elements
- ▶ Dynamic array is array with variable number of elements: actually a pointer and a variable indicating n° of elements

```
int n = 5;  
int* arr = (int*)malloc(n*sizeof(int));
```

- ▶ Insert an element into dynamic array:

```
pos = 2; val = 25;  
arr1 = (int*)malloc((n+1)*sizeof(int));  
memcpy(arr1, arr, pos*sizeof(int));  
memcpy(arr1+pos+1, arr+pos, (n-pos)*sizeof(int));  
arr1[pos] = val;  
free(arr);  
arr = arr1; n++;
```

- ▶ Similarly for element removal

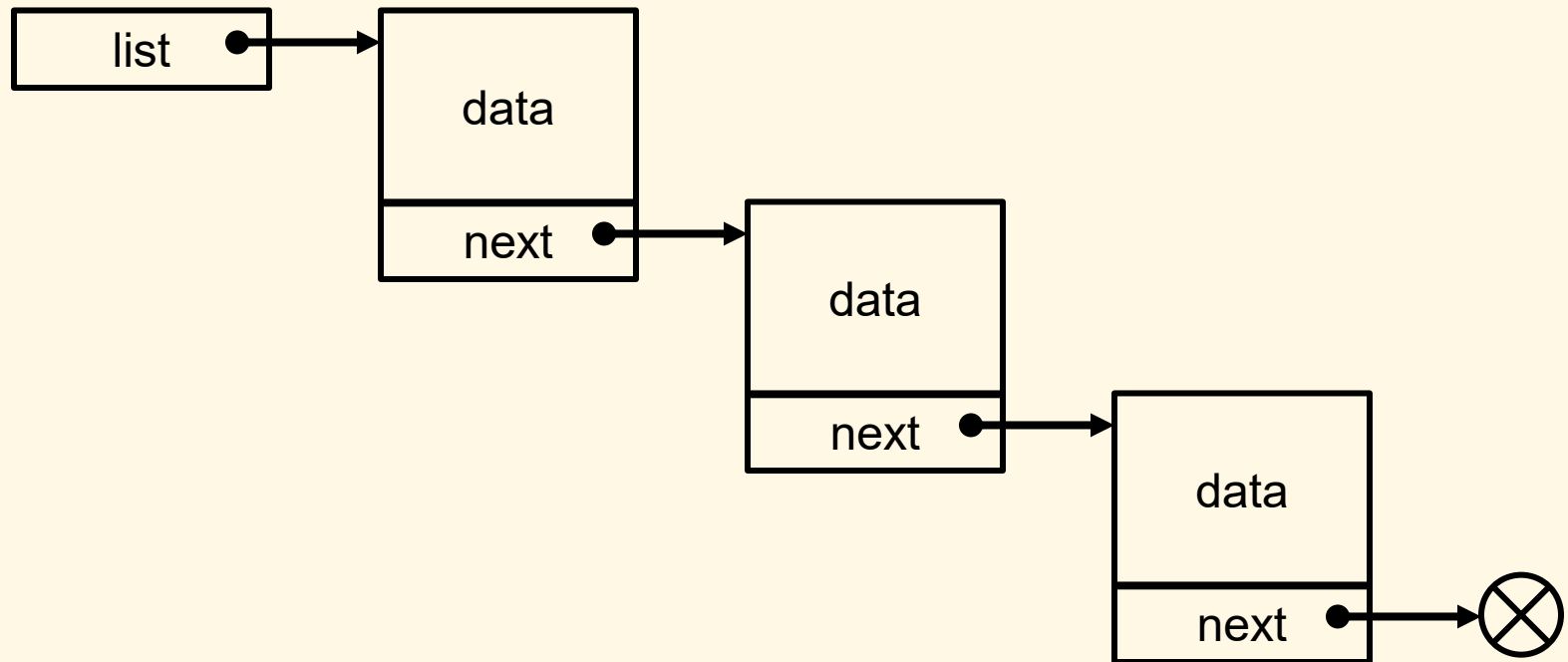


Overview

- ▶ Above example shows that dynamic arrays are slow in inserting/removing elements because of moving large memory areas, especially when there are many elements
→ necessity of a more flexible data structures
- ▶ Frequently used data structures:
 - ▶ Stack
 - ▶ Queue
 - ▶ Linked list
 - ▶ Dynamic array (vector)
 - ▶ Map, dictionary, hash table
 - ▶ Set
 - ▶ Tree
 - ▶ Graph

Linked list

- ▶ Group of nodes linked together by pointers and represent a sequence of elements. For presentation:
 - ▶ A pointer to the first node (or NULL for empty list)
 - ▶ Each node includes 2 members: data, and a pointer to the next node
 - ▶ The `next` pointer of last element is NULL



Declaration of linked list

► Example for `int` elements:

```
typedef int LLIST_DATA;
```

```
struct _LIST_ELEM;
```

```
typedef struct _LIST_ELEM LIST_ELEM, *LLIST;
```

```
struct _LIST_ELEM {  
    LLIST_DATA data;  
    LIST_ELEM* next;  
};
```

Operations of linked list

► Necessary functions:

- `void llInit(LLIST* l);`
- `void llInsertHead(LLIST* l, LLIST_DATA data);`
- `void llInsertTail(LLIST* l, LLIST_DATA data);`
- `void llInsertAfter(LLIST* l, LIST_ELEM* a, LLIST_DATA data);`
- `void llDeleteHead(LLIST* l);`
- `void llDeleteTail(LLIST* l);`
- `void llDeleteAfter(LLIST* l, LIST_ELEM* a);`
- `void llDeleteAll(LLIST* l);`
- `LIST_ELEM* llSeek(LLIST l, int i);`
- `void llForEach(LLIST l, LLCALLBACK func, void* user);`
- `int llLength(LLIST l);`

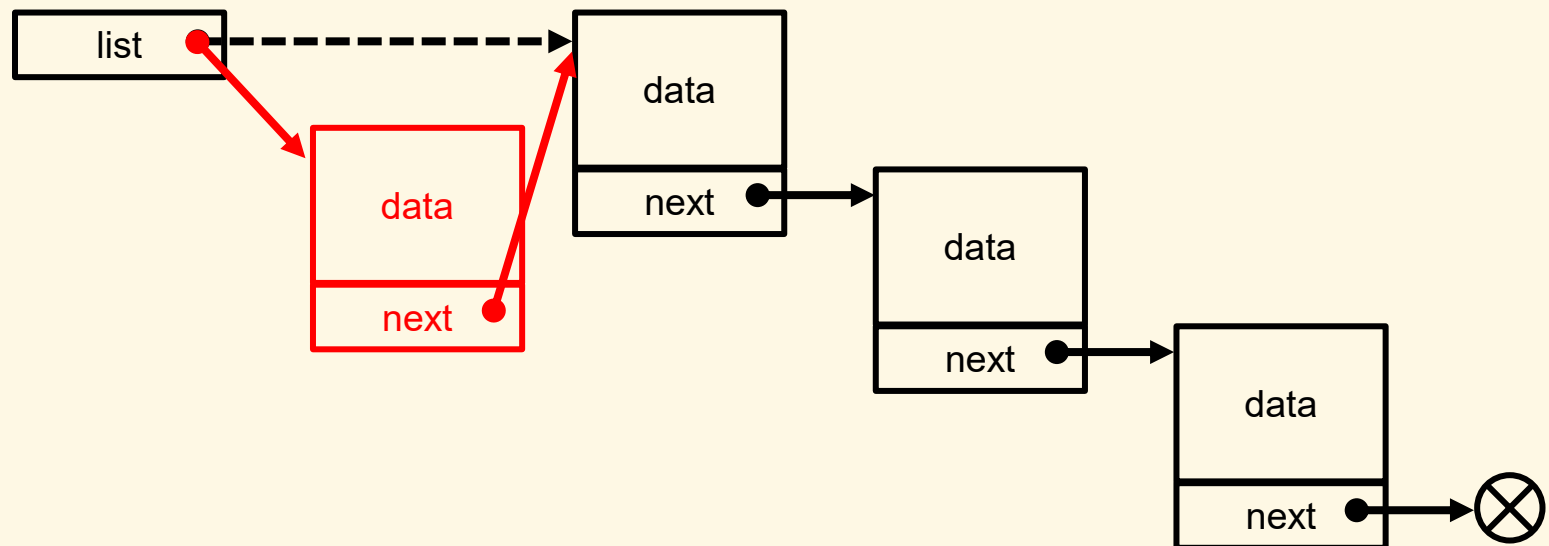
Initialization

```
void llInit(LLIST* l) {  
    *l = NULL;  
}
```

- Initialized to NULL → empty list

Insert element to head of a list

```
void llInsertHead(LLIST* l, LLIST_DATA data) {  
    LIST_ELEM* e = (LIST_ELEM*)malloc(sizeof(LIST_ELEM));  
    e->data = data;  
    e->next = *l;  
    *l = (LLIST)e;  
}
```

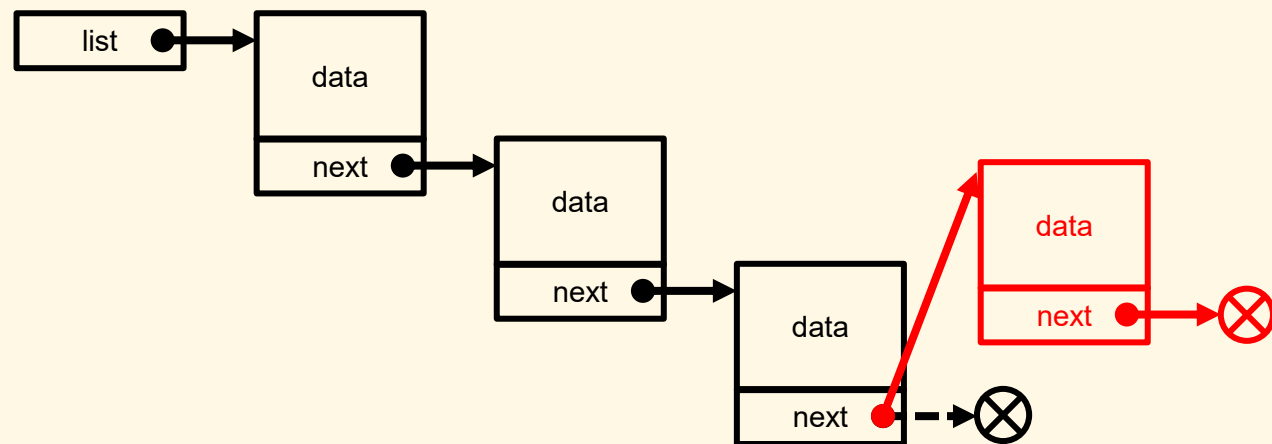


Insert element to tail of a list

```
void llInsertTail(LLIST* l, LLIST_DATA data) {  
    LIST_ELEM* e = (LIST_ELEM*)malloc(sizeof(LIST_ELEM));  
    e->data = data;  
    e->next = NULL;
```

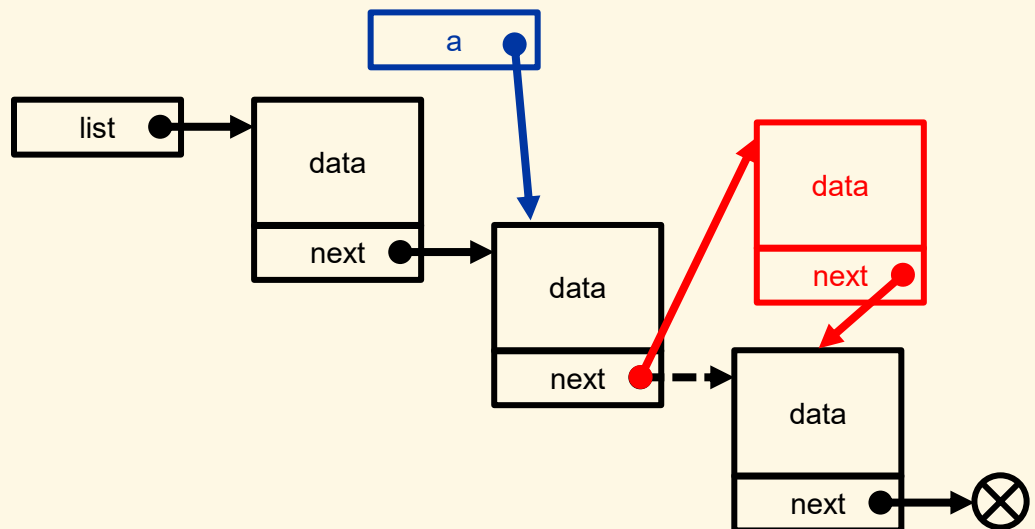
```
    if (!*l) {  
        *l = e;  
        return;  
    }
```

```
    LIST_ELEM* p;  
    for (p = *l; p->next; p = p->next) ;  
    p->next = e;  
}
```



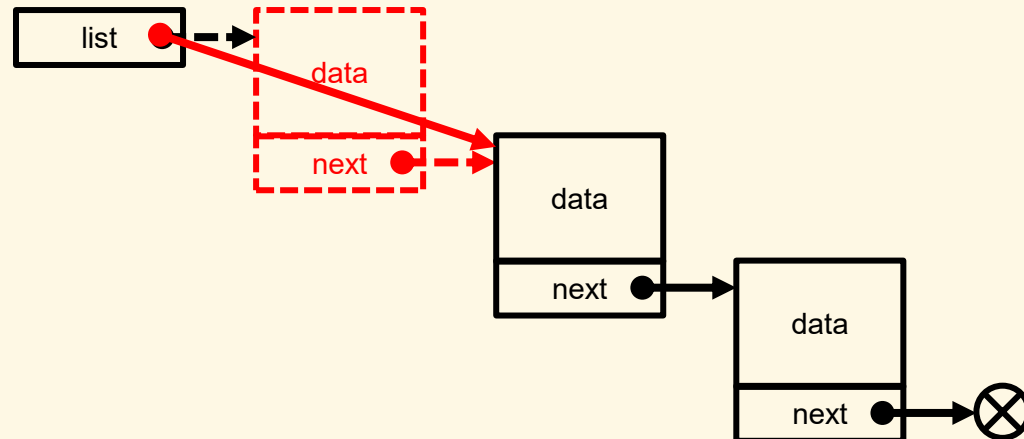
Insert elements in the middle

```
void llInsertAfter(LLIST* l, LIST_ELEM* a, LLIST_DATA data) {  
    if (!a) return;  
  
    LIST_ELEM* e = (LIST_ELEM*)malloc(sizeof(LIST_ELEM));  
    e->data = data;  
    e->next = a->next;  
    a->next = (LLIST)e;  
}
```



Delete first element

```
void llDeleteHead(LLIST* l) {  
    if (!*l) return;  
  
    LIST_ELEM* p = (*l)->next;  
    free(*l);  
    *l = (LLIST)p;  
}
```



Delete last element

```
void llDeleteTail(LLIST* l) {
```

```
    if (!*l) return;
```

```
    if (!(*l)->next) {
```

```
        free(*l);
```

```
        *l = NULL;
```

```
        return;
```

```
    }
```

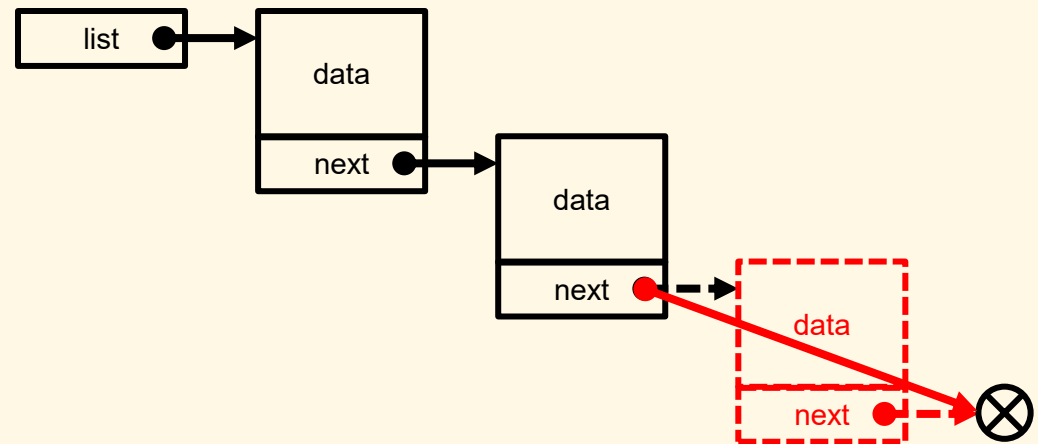
```
    LIST_ELEM* p;
```

```
    for (p = *l; p->next->next; p = p->next) ;
```

```
    free(p->next);
```

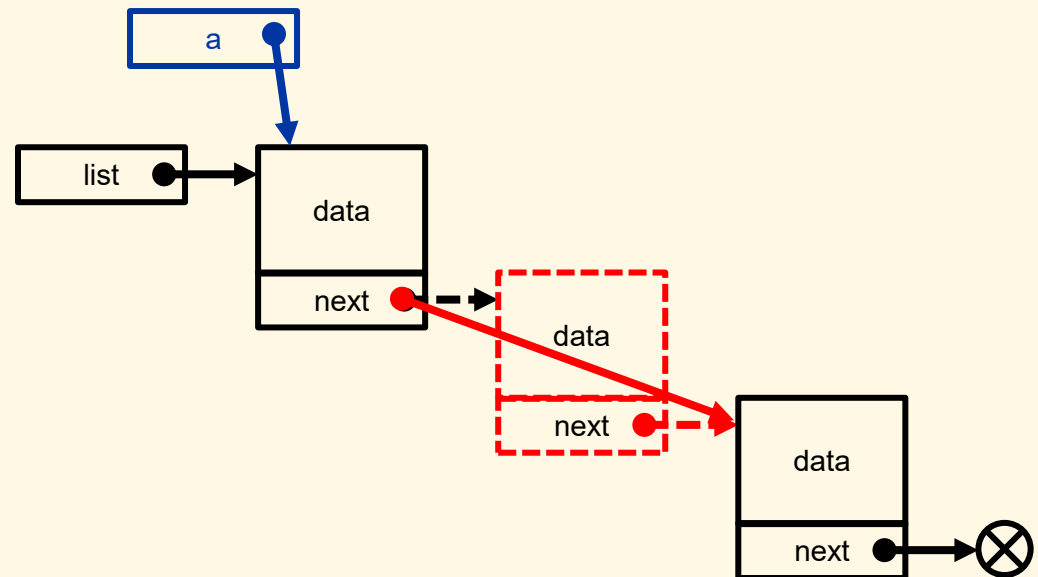
```
    p->next = NULL;
```

```
}
```



Delete elements in the middle

```
void llDeleteAfter(LLIST* l, LIST_ELEM* a) {  
    if (!a || !a->next) return;  
  
    LIST_ELEM* p = a->next;  
    a->next = p->next;  
  
    free(p);  
}
```



Delete all elements

```
void llDeleteAll(LLIST* l) {  
    LIST_ELEM *p, *i;  
    for (i = *l; i; i = p) {  
        p = i->next;  
        free(i);  
    }  
    *l = NULL;  
}
```

Count the number of elements

```
int llLength(LLIST l) {  
    int c;  
    for (c = 0; l; c++)  
        l = l->next;  
    return c;  
}
```

Iterate a list

- Find the i^{th} element of a list

```
LIST_ELEM* llSeek(LLIST l, int i) {  
    for (; i>0 && l; i--)  
        l = l->next;  
    return (LIST_ELEM*)l;  
}
```

- Do some operation to all elements of a list

```
typedef void (*LLCALLBACK)(LLIST_DATA, void*);  
  
void llForEach(LLIST l, LLCALLBACK func, void* user) {  
    for (; l; l = l->next)  
        func(l->data, user);  
}
```


Usage example

```
#include <stdio.h>
#include "llist.h"

void printList(LLIST l) {
    printf("%d elements: ( ",
           llLength(l));
    for (; l; l = l->next)
        printf("%d ", l->data);
    printf(") \n");
}

void listSum(int data, void* user) {
    int* pS = (int*)user;
    *pS += data;
}

int main() {
    LLIST l;
    LIST_ELEM* p;
    int i, s;

    llInit(&l);

    for (i = 0; i < 5; i++) {
        llInsertTail(&l, i);
        llInsertHead(&l, -i);
    }
    printList(l);

    p = llSeek(l, 1);
    llDeleteAfter(&l, p);
    llDeleteHead(&l);
    llDeleteTail(&l);
    printList(l);

    s = 0;
    llForEach(l, listSum, (void*)&s);
    printf("Sum of values: %d\n", s);

    llDeleteAll(&l);
    printList(l);

    return 0;
}
```

Properties of linked lists

- ▶ Advantages/drawbacks of linked lists compared to dynamic arrays:
 - + Flexible: easy to insert/remove elements, sort or change the order of elements without moving them in memory
 - + No need to allocate a large consecutive memory area to store all elements
 - Need additional memory to store `next` pointers
 - Sequential access: impossible to directly access arbitrary elements (or random access), but need to iterate from the beginning
- ▶ Beside basic form (singly linked list), there are other forms: doubly linked list, ordered linked list, stack, queue, cycle linked list,... and can be implemented in many different ways depending on specific problems

Exercises

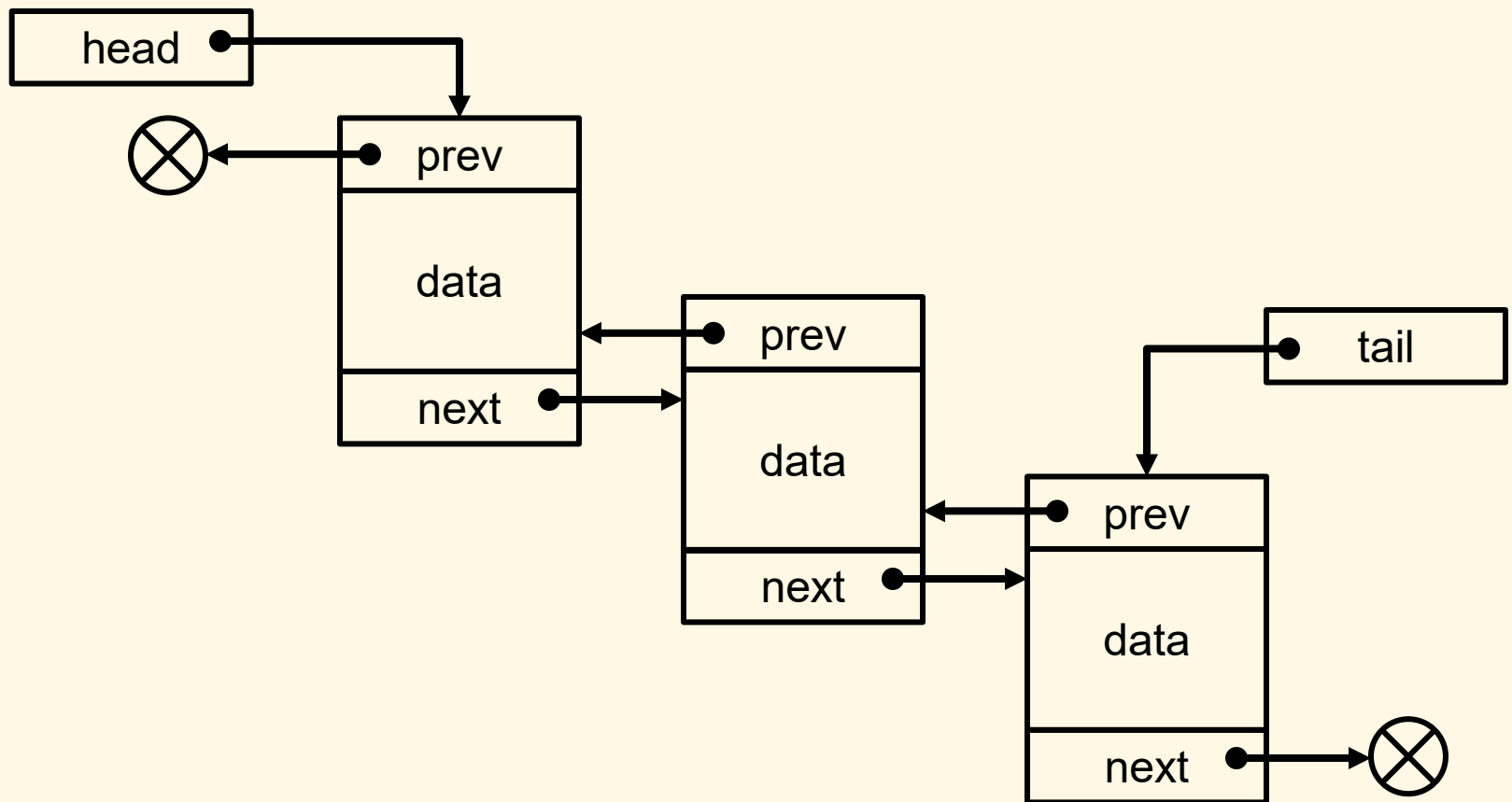
1. Calculate the average value of a linked list by two ways: iteration, using `llForEach()` function.
2. Write `llConvert(int* arr, int count)` function to convert an array into a linked list without using functions from the library.
3. Given two linked lists `l1` and `l2`, write `llInsertListAfter(&l1, l2, p)` function to insert `l2` into `l1` after element `p`.
4. Implement a linked list that could be used for different types (Hint: make data field dynamic).

Doubly Linked Lists

Overview

- ▶ Singly linked list (SLL) only allows to iterate in forward direction
 - ▶ Each element has only a pointer to the next one
 - ▶ Impossible to find an element given the following one
- ▶ Doubly linked list (DLL):
 - ▶ Possible to move forward and backward at any time
 - ▶ Each element has a 2nd pointer to the previous one

How DLLs work



Exercise

1. Implement a doubly linked list.

Homework

1. Write `llReverse (LLIST* l)` function to reverse the order of a linked list without creating new nodes.
2. Same as Prob. 1, but do recursively.